FEMaster User Manual

Finite Elements in Mechanical Analysis and Simulation for Technical Engineering Research

Version 2.8.29

20 September 2026

Contents

1 Overview **1**

1.1 Introduction . . . 1
1.2 Design philosophy and implementation . . . 1
1.3 Input format and compatibility . . . 2
1.4 Supported element families . . . 2
1.5 Supported analysis families . . . 3
1.6 Degree-of-freedom convention . . . 4
1.7 Units . . . 4
1.8 Manual organization . . . 5

2 Getting Started **6**

2.1 Installation . . . 6
2.2 Building FEMaster . . . 6
2.2.1 Available build presets . . . 7
2.3 Running FEMaster . . . 8
2.4 First example: linear static truss . . . 8
2.5 Result files . . . 9
2.6 Built-in documentation . . . 9

A Mathematical Derivations **11**

A.1 Element stiffness . . . 11
A.2 Compliance . . . 11
A.3 Density penalization . . . 12
A.4 Orientation-dependent stiffness . . . 12
A.5 Voigt transformation . . . 12
A.6 Orientation sensitivity . . . 13
A.7 Rotation derivatives . . . 13

1 Overview

1.1 Introduction

FEMaster (*Finite Elements in Mechanical Analysis and Simulation for Technical Engineering Research*) is an open-source finite-element solver for structural mechanics, implemented in modern C++. It is designed for engineering applications, scientific research, and education, providing a common framework for the numerical analysis of structures ranging from simple mechanical components to large and complex finite-element models. Its capabilities include different structural element formulations, static and dynamic analyses, eigenvalue problems, and nonlinear structural behavior.

The project originated during the author's university studies, following an internship at MTU where working with CalculiX provided the motivation to develop an independent finite-element solver. Rather than simply reproducing existing functionality, the objective was to implement established numerical methods within a modern, maintainable, and accessible codebase. FEMaster is intended to serve both as a practical engineering tool and as a platform for studying, developing, and verifying finite-element methods. Particular emphasis is placed on numerical transparency, computational performance, and a broad selection of element formulations, allowing users to investigate both the physical response of a structure and the numerical methods used to obtain it.

1.2 Design philosophy and implementation

The development of FEMaster is guided by three closely related objectives: numerical transparency, maintainability, and computational efficiency. Numerical transparency means that the methods used by the solver should be accessible and understandable, rather than hidden behind an opaque implementation. The source code is organized around clearly defined mechanical and numerical concepts, with an emphasis on readable implementations, meaningful interfaces, and documentation. Mathematical formulations and numerical procedures are described throughout this manual where appropriate, providing a connection between the theoretical foundations of finite-element analysis and their implementation in the solver. This is particularly relevant for educational applications, method development, and the verification of numerical results.

Maintainability is supported by a modular C++ architecture that separates model definitions, element formulations, material behavior, analysis procedures, and numerical solution methods. This separation allows individual components to be developed and examined without requiring detailed knowledge of the entire application. It also facilitates the implementation of additional element formulations, constitutive models, and analysis capabilities. The intention is to maintain a codebase that remains approachable as the solver grows in complexity, enabling contributions and modifications without sacrificing the clarity of existing implementations.

Computational efficiency is addressed through parallel execution, efficient numerical algorithms, and established high-performance libraries. FEMaster uses Eigen for linear algebra operations and Spectra for eigenvalue computations, while Intel oneMKL provides high-performance multithreaded CPU solution capabilities. GPU acceleration is supported through NVIDIA CUDA and cuDSS, allowing computationally demanding solution tasks to be executed on compatible graphics hardware. The available numerical backends depend on the platform and build configuration. These technologies are intended to provide efficient

solutions for both small models, where computational overhead can be significant relative to the actual problem size, and large systems, where memory requirements and the performance of sparse numerical operations become increasingly important.

1.3 Input format and compatibility

FEMaster uses a unified keyword-based input format derived from the conventions established by Abaqus and also used by CalculiX. A finite-element model is described through a text file containing definitions for geometry, element connectivity, materials, structural properties, loads, boundary conditions, and analysis procedures. The format is particularly suitable for automated model generation, parameter studies, and reproducible numerical investigations, while remaining sufficiently readable for small models to be created and modified manually.

The input language supports a subset of the Abaqus keyword syntax together with FEMaster-specific extensions. Both forms are interpreted within the same input language and can generally be combined in a single input deck. This allows existing Abaqus-style models to be used where their definitions are supported, while additional FEMaster keywords provide access to functionality beyond the conventional syntax. Full compatibility with arbitrary Abaqus input files is not implied, since the supported keywords, parameters, element formulations, and numerical procedures depend on the capabilities implemented in FEMaster. The Keyword Reference documents these capabilities and presents alternative Abaqus-compatible and FEMaster-specific syntax wherever applicable.

Several extensions introduce concepts inspired by preprocessing environments such as Altair HyperWorks. In particular, loads and supports can be organized into named collectors that are defined independently of individual analyses. These collectors can subsequently be referenced by analysis steps, allowing different combinations of loads and boundary conditions to be applied without repeatedly defining the same conditions. Additional keywords provide explicit control over analysis procedures, constraint handling, solver selection, and the use of CPU or GPU acceleration. Together, these extensions are intended to make input decks easier to organize, reuse, and inspect, particularly when a model contains several loading configurations or analysis procedures.

1.4 Supported element families

FEMaster provides a range of structural finite elements for modelling one-, two-, and three-dimensional mechanical behavior. The supported families include truss, beam, shell, continuum solid, and point elements, each offering different capabilities and levels of geometric detail. The choice of element formulation determines which deformation modes can be represented, which physical properties are required, and how accurately a particular structural response can be captured.

An important characteristic of FEMaster is its support for six mechanical degrees of freedom per node, consisting of three translations and three rotations. Individual elements activate only the degrees of freedom required by their formulation. In particular, beam and shell elements use independent rotational degrees of freedom and are implemented as dedicated structural elements. Unlike the approach used for many conventional beam and shell formulations in CalculiX, these elements are not internally expanded into three-dimensional continuum elements. Instead, their stiffness, internal forces, and other mechanical quantities are evaluated directly from their respective structural formulations.

Continuum solid elements represent the full three-dimensional deformation and stress state of a body. FEMaster supports tetrahedral, hexahedral, wedge, and pyramid elements with different interpolation orders and integration schemes, including reduced integration and enhanced formulations. They are particularly suitable for complex geometries, local stress concentrations, and regions where three-dimensional effects are important.

Beam and truss elements provide computationally efficient representations of slender structures. Truss elements carry axial forces, while beam elements additionally represent bending, torsion, and transverse shear, depending on the formulation. Their cross-sectional behavior is defined through geometric and

material properties rather than an explicitly meshed cross-section. FEMaster supports both classical beam formulations and geometrically exact formulations capable of representing finite rotations, making them suitable for frames, stiffeners, spars, and other structural members.

Shell elements describe thin-walled structures through a two-dimensional reference surface and associated thickness properties. They capture membrane deformation, bending, and transverse shear without requiring a volume mesh through the thickness. FEMaster includes shell formulations based on the Mixed Interpolation of Tensorial Components (MITC) approach with finite-rotation kinematics, making them suitable for plates, panels, curved shells, and lightweight structures.

Point elements introduce concentrated properties such as masses, rotary inertias, and spring stiffnesses at individual nodes. They allow attached equipment, payloads, and simplified elastic supports to be represented without explicitly modelling their geometry.

The different element families can be combined within a single finite-element model, allowing an appropriate balance between computational efficiency and mechanical detail. The available element types, their formulations, numerical characteristics, and limitations are documented in Chapter **??**.

1.5 Supported analysis families

FEMaster supports several analysis procedures covering static structural behavior, stability, free vibration, and time- or frequency-dependent response. An analysis procedure determines the governing equations to be solved, the required system matrices, and the numerical strategy used to obtain the solution. The same finite-element model can be evaluated using different procedures, allowing structural stiffness, deformation, stability, and dynamic characteristics to be investigated without redefining the underlying geometry and material properties. The currently supported analysis families are summarized below.

FEMaster analysis	**Description**
`LINEARSTATIC`	Linear static equilibrium analysis under prescribed loads and boundary conditions. The procedure determines displacements, reactions, strains, and stresses using a linearized structural model. It is suitable for stiffness evaluation, load-path investigations, and structural assessments in which nonlinear effects can be neglected.
`LINEARSTATICTOPO`	Linear static analysis with topology-dependent element properties. Element-wise density values, penalization, and optional orientation Fields allow structural responses to be evaluated for different material distributions, providing a basis for topology-related investigations and optimization workflows.
`NONLINEARSTATIC`	Incremental static equilibrium analysis accounting for nonlinear structural behavior. The solution is obtained through iterative equilibrium corrections, with support for load-controlled increments and arc-length path following. Depending on the element formulations and model definitions, nonlinear effects may include large displacements, finite rotations, material nonlinearities, and contact.
`EIGENFREQ`	Free-vibration analysis based on the generalized eigenvalue problem of the structural stiffness and mass matrices. The procedure computes natural frequencies and their associated mode shapes, allowing the dynamic characteristics of a structure to be investigated independently of a prescribed time-dependent excitation.
`LINEARBUCKLING`	Linearized stability analysis using the structural stiffness and the geometric stiffness associated with a reference loading state. The resulting eigenvalues describe critical load factors within the linearized buckling approximation, while the corresponding eigenvectors identify possible instability modes.

FEMaster analysis	Description
`LINEARTRANSIENT`	Linear time-domain dynamic analysis including inertial and optional damping effects. The equations of motion are integrated using an implicit Newmark scheme, allowing the response to time-dependent loading, initial velocities, and transient excitation to be evaluated over a prescribed time interval.
`LINEARHARMONIC`	Linear steady-state response to harmonic excitation. The procedure solves the frequency-dependent complex equation system directly at specified excitation frequencies, providing displacement and other supported response amplitudes together with their phase information. Optional damping can be included through Rayleigh damping.

The analysis procedures differ not only in their numerical solution methods but also in the physical assumptions underlying their results. Linear static analysis and linearized eigenvalue procedures are generally efficient and suitable for a wide range of engineering investigations, provided their assumptions are appropriate for the problem. Nonlinear static analysis extends the equilibrium formulation to situations in which the structural response changes with deformation, loading, or contact conditions, but typically requires incremental loading and iterative convergence. Dynamic procedures additionally account for mass and inertia, either by integrating the response over time or by evaluating the steady-state response at selected excitation frequencies. These distinctions should be considered when selecting a procedure, since the most computationally demanding analysis is not necessarily the most appropriate one for a particular engineering question.

Both FEMaster-specific analysis definitions and supported Abaqus procedure keywords are available within the input language. The corresponding Abaqus procedures include `*STATIC`, `*FREQUENCY`, `*BUCKLE`, `*DYNAMIC`, and `*STEADY STATE DYNAMICS`. Their accepted parameters, available numerical controls, and relationship to the FEMaster analysis types are documented in Chapter **??**.

1.6 Degree-of-freedom convention

FEMaster uses a generalized six-component nodal convention consisting of three translations and three rotations:

`Ux, Uy, Uz, Rx, Ry, Rz`.

The first three components represent translational displacements along the global coordinate axes, while the last three represent rotational degrees of freedom about those axes. This convention is used for generalized nodal loads, supports, coupling definitions, initial velocities, and selected result Fields. It does not imply that every node necessarily contributes six degrees of freedom to the global equation system. The active degrees of freedom depend on the connected element formulations and associated model definitions. For example, continuum solid elements generally require only translations, whereas beam and shell elements additionally use rotations. This distinction permits different structural element families to be represented within a common generalized framework without introducing unnecessary degrees of freedom throughout the model.

The six-component nodal convention must not be confused with the six-component engineering stress and strain convention, which represents the independent components of symmetric second-order tensors rather than generalized displacements. Stress and strain component ordering is documented in Section **??**.

1.7 Units

FEMaster does not prescribe a particular system of units or perform automatic unit conversions. All geometric dimensions, material properties, loads, masses, and numerical parameters must therefore be expressed using a consistent system of units. A common choice for structural analysis is millimetres,

newtons, tonnes, and seconds, resulting in stresses and elastic moduli expressed in megapascals. Other systems, such as metres, newtons, kilograms, and seconds, are equally valid when applied consistently. Particular attention is required for density, rotary inertia, dynamic loading, damping coefficients, thermal expansion, and stiffness-related parameters, since inconsistent units may produce numerical results that appear plausible despite having no meaningful physical interpretation. Responsibility for unit consistency remains with the user, and the solver interprets all supplied quantities according to the governing finite-element equations.

1.8 Manual organization

This manual begins with Chapter 2, which introduces installation, command-line execution, and a first complete calculation, including the interpretation and verification of its results. Chapter **??** then explains the fundamental concepts of the FEMaster model architecture and the relationships between its principal objects. Building on these concepts, Chapter **??** provides the detailed input language reference, including supported keywords, parameters, data records, and alternative Abaqus-compatible definitions. The individual modelling and analysis concepts are brought together in Chapter **??**, which presents complete applications and demonstrates how the available functionality can be used to investigate engineering problems. Additional mathematical background, numerical formulations, and derivations are provided in the appendices.

2 Getting Started

This chapter explains how to obtain FEMaster, compile it from source, and perform a first finite-element analysis. FEMaster is a command-line application that reads a keyword-based input file, performs the requested calculations, and writes the resulting data to output files. No graphical interface is required, allowing the solver to be used both interactively and as part of automated simulation workflows.

2.1 Installation

FEMaster is available for Linux, Windows, and macOS. The source code, precompiled releases, and additional project information are hosted on GitHub at `https://github.com/Luecx/FEMaster`.

The simplest way to get started is to download a precompiled version from `https://github.com/Luecx/FEMaster/releases`. Where available, release archives contain executables for the corresponding operating system and architecture. After extracting the archive, the executable can be launched directly from a terminal. Depending on the selected release, additional runtime libraries may be required.

Alternatively, FEMaster can be compiled from source. This allows the solver to be configured for the available hardware, including optional multithreaded CPU and GPU acceleration, and provides access to the latest development version. The source code can be obtained through Git:

```
git clone https://github.com/Luecx/FEMaster.git
cd FEMaster
```

2.2 Building FEMaster

FEMaster uses CMake to manage its compilation and external dependencies. The repository provides predefined build configurations, called *presets*, for the supported operating systems and numerical backends. A preset selects the compiler configuration, optimization settings, and optional libraries required for a particular build.

From the repository root, configuring and compiling FEMaster requires only two commands. For example, a basic Linux build can be created using:

```
cmake --preset linux
cmake --build --preset linux -j8
```

The name `linux` is the selected preset and can be replaced by any of the available presets listed below. The first command configures the project and prepares its dependencies, while the second compiles the executable. The optional argument `-j8` specifies eight parallel build jobs. Subsequent builds generally require only the second command, unless the build configuration changes.

Compilation requires CMake 3.24 or newer, Git, and a compiler supporting C++17. The presets using

Ninja additionally require Ninja to be installed. Standard dependencies, including Eigen, Spectra, argparse, and LZ4, are downloaded automatically by CMake during configuration. Optional numerical libraries such as Intel oneMKL, NVIDIA CUDA, and cuDSS must be installed separately when their corresponding backends are enabled. On macOS, Apple Accelerate is provided by the system SDK.

2.2.1 Available build presets

The predefined presets provide different combinations of CPU parallelization, sparse solvers, and GPU acceleration. All regular presets produce optimized Release builds, with the exception of the profiling configuration. The available presets are summarized below.

Preset	Configuration
Linux	
`linux`	Standard CPU build.
`linux-omp`	CPU build with OpenMP parallelization.
`linux-profile`	CPU build with profiling and debugging symbols.
`linux-mkl`	CPU build with sequential Intel oneMKL.
`linux-mkl-omp`	Threaded Intel oneMKL with OpenMP.
`linux-cuda`	NVIDIA CUDA acceleration.
`linux-cuda-cudss`	CUDA with NVIDIA cuDSS sparse direct solving.
`linux-cuda-mkl`	CUDA with sequential Intel oneMKL.
`linux-cuda-mkl-omp`	CUDA with threaded Intel oneMKL and OpenMP.
`linux-full`	CUDA, cuDSS, threaded Intel oneMKL, and OpenMP.
Windows	
`windows-msvc`	Standard CPU build using MSVC.
`windows-msvc-cuda`	MSVC build with CUDA acceleration.
`windows-msvc-cuda-cudss`	MSVC build with CUDA and cuDSS.
`windows-intel`	Standard CPU build using IntelLLVM.
`windows-intel-mkl`	IntelLLVM with sequential Intel oneMKL.
`windows-intel-mkl-omp`	IntelLLVM with threaded Intel oneMKL and OpenMP.
macOS	
`macos`	Standard CPU build.
`macos-omp`	CPU build with OpenMP parallelization.
`macos-accelerate`	CPU build with Apple Accelerate.
`macos-omp-accelerate`	OpenMP with Apple Accelerate.

The available configurations depend on the installed toolchains and hardware. Windows MSVC presets use Visual Studio, while the IntelLLVM presets require the Intel compiler toolchain. CUDA configurations require a compatible NVIDIA GPU and CUDA installation, and cuDSS configurations additionally require the cuDSS library. Apple Accelerate is available only on macOS.

CMake places the compiled executable in the build directory associated with the selected preset, typically `build/<preset>/bin`. Multi-configuration generators, such as the Visual Studio presets, place the Release executable in `build/<preset>/bin/Release`. The executable is named `FEMaster` on Linux and macOS and `FEMaster.exe` on Windows.

The exact preset definitions and additional configuration options can be found in `CMakePresets.json` and `CMakeLists.txt`.

2.3 Running FEMaster

FEMaster is executed by passing the path of an input deck to the command-line application. Input files conventionally use the `.inp` extension and contain the model definition together with the requested analysis procedures. Both supported Abaqus keywords and FEMaster-specific extensions are interpreted by the same parser, so no explicit input-format selection is required.

If the executable is available on the system path, a model can be executed using:

```
FEMaster example.inp
```

Alternatively, the executable can be called using its location in the build directory. For example, from the repository root after compiling with the `linux` preset:

```
./build/linux/bin/FEMaster example.inp
```

Several optional command-line arguments control execution and result storage. The most commonly used options are:

Argument	Description
`-ncpus N`	Set the number of CPU threads available to the solver.
`-output FILE`	Specify an alternative output file base name or path.
`-output-format`	Select one or more result formats: `res`, `frd`, or `femr`.

For example, the following command limits execution to eight CPU threads and requests FRD and FEMR output:

```
FEMaster example.inp --ncpus 8 --output-format frd femr
```

The selected numerical backend is configured through the input deck and must be supported by the compiled executable. The complete command-line interface can be inspected using `-help`.

2.4 First example: linear static truss

As a first calculation, consider a straight truss element subjected to an axial tensile force. The model consists of two nodes connected by a single element with a length of 1000 mm and a cross-sectional area of 100 mm^2. The material is linear elastic steel with Young's modulus of 210000 MPa. One end is fixed, while the opposite end is allowed to move only in the axial direction and is subjected to a force of 1000 N.

Save the following input deck as `example.inp`:

Syntax

```
*HEADING
First FEMaster example

*NODE
1, 0., 0., 0.
2, 1000., 0., 0.

*ELEMENT, TYPE=T3, ELSET=BAR
1, 1, 2

*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3

*TRUSS SECTION, ELSET=BAR, MATERIAL=STEEL
100.

*STEP, NAME=AXIAL_TENSION
*STATIC
*BOUNDARY
1, 1, 3, 0.
2, 2, 3, 0.
*CLOAD
2, 1, 1000.
*END STEP
```

The example combines FEMaster-specific element and section definitions with Abaqus-style analysis keywords. Both are supported within the unified input language. To perform the analysis, execute:

```
FEMaster example.inp
```

FEMaster reads the model, assembles the finite-element system, solves the static equilibrium equations, and writes the calculated results to output files. The expected axial displacement at the loaded node is approximately 0.047619 mm, providing a simple reference value for checking the calculation.

2.5 Result files

By default, FEMaster writes the results of an analysis using the input filename as the output base name. Running `example.inp` therefore produces `example.res` and `example.frd`. An additional binary result format, `.femr`, is available through the `-output-format` option.

The `.res` format stores numerical result Fields in a text-based representation, while `.frd` provides results in the CalculiX/CGX format for visualization and postprocessing. The `.femr` format provides structured binary storage with compression for more efficient handling of larger result datasets. The available result quantities, output controls, and file conventions are described in Section **??**.

2.6 Built-in documentation

In addition to this manual, FEMaster provides a command-line documentation mode for inspecting the keywords and input variants recognized by the executable. This can be useful when working with an unfamiliar command or checking which syntax forms are supported by a particular build.

Documentation mode is activated using `-document`, followed by an action specifying the requested information:

```
FEMaster --document --doc-list
FEMaster --document --doc-show CLOAD
FEMaster --document --doc-variants ELASTIC
FEMaster --document --doc-search SECTION
```

These commands list available keywords, display the definition of a particular command, inspect its accepted data variants, or search the command registry. Documentation mode does not require an input deck and does not perform a finite-element analysis. For detailed explanations, modelling conventions, and practical examples, the Keyword Reference remains the primary source of information.

A Mathematical Derivations

This appendix derives the topology-related quantities written by FEMaster. The keyword reference sections describe the DSL commands, while this appendix gives the mathematical background for compliance, density sensitivities, and orientation sensitivities.

A.1 Element stiffness

For a linear elastic finite element, the unmodified element stiffness is

$$K_{e,0} = \int_{\Omega_e} B^T D B \, d\Omega.$$

B is the strain-displacement matrix, D is the constitutive matrix, and Ω_e is the physical element domain. With an isoparametric mapping from natural coordinates $r = (\xi, \eta, \zeta)$, the integral is evaluated as

$$K_{e,0} = \int_{\Omega'_e} B(r)^T D B(r) \, |\det J(r)| \, d\xi \, d\eta \, d\zeta.$$

For numerical integration with quadrature points r_q and weights w_q, this becomes

$$K_{e,0} \approx \sum_{q=1}^{Q} w_q B(r_q)^T D B(r_q) \, |\det J(r_q)|.$$

A.2 Compliance

For a linear static analysis,

$$Ku = f.$$

The external work measure commonly used for topology optimization is the compliance

$$C = f^T u.$$

Using the equilibrium equation $f = Ku$, this can also be written as

$$C = u^T K u.$$

Because the global stiffness matrix is assembled from element stiffness matrices, compliance can be decomposed into element contributions:

$$C = \sum_{e=1}^{n_e} u_e^T K_e u_e.$$

This expression is the basis for element-wise topology sensitivities. u_e is the element displacement vector extracted from the global solution.

A.3 Density penalization

In a density-based topology formulation, an element density variable ρ_e scales the stiffness contribution. With a penalization exponent p,

$$K_e(\rho_e) = \rho_e^p K_{e,0}.$$

The exponent p makes intermediate densities less efficient and therefore encourages designs closer to a solid-void distribution.

The element compliance contribution is

$$C_e(\rho_e) = u_e^T K_e(\rho_e) u_e = \rho_e^p u_e^T K_{e,0} u_e.$$

If u_e is held fixed for the local sensitivity expression, the derivative is

$$\frac{\partial C_e}{\partial \rho_e} = p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

Depending on the optimization convention, the objective derivative may be reported with the opposite sign because minimizing compliance through the equilibrium equation often uses the adjoint result

$$\frac{dC}{d\rho_e} = -u_e^T \frac{\partial K_e}{\partial \rho_e} u_e = -p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

The sign convention is therefore defined together with the optimizer interface and the Field name written by the analysis.

A.4 Orientation-dependent stiffness

For orientation-based topology or material steering, the constitutive matrix is rotated before the element stiffness is evaluated. Let the material orientation be represented by three angles

$$\alpha = (\alpha_1, \alpha_2, \alpha_3).$$

The elementary rotation matrices are

$$R_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \\ 0 & \sin\alpha_1 & \cos\alpha_1 \end{bmatrix},$$

$$R_2 = \begin{bmatrix} \cos\alpha_2 & 0 & \sin\alpha_2 \\ 0 & 1 & 0 \\ -\sin\alpha_2 & 0 & \cos\alpha_2 \end{bmatrix}, \qquad R_3 = \begin{bmatrix} \cos\alpha_3 & -\sin\alpha_3 & 0 \\ \sin\alpha_3 & \cos\alpha_3 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

For the convention used here, the combined rotation is

$$R = R_1 R_2 R_3.$$

The multiplication order must be used consistently. Different conventions can describe the same physical orientation with different angle triples.

A.5 Voigt transformation

The strain vector uses engineering shear strains,

$$\varepsilon = \begin{bmatrix} \varepsilon_{xx} & \varepsilon_{yy} & \varepsilon_{zz} & 2\varepsilon_{yz} & 2\varepsilon_{xz} & 2\varepsilon_{xy} \end{bmatrix}^T,$$

and the stress vector is

$$\sigma = \begin{bmatrix} \sigma_{xx} & \sigma_{yy} & \sigma_{zz} & \sigma_{yz} & \sigma_{xz} & \sigma_{xy} \end{bmatrix}^T.$$

The strain transformation is written as

$$\varepsilon_m = T(R)\varepsilon_g,$$

where T is the 6×6 Voigt transformation matrix built from the entries of R. The rotated constitutive matrix used in global strain coordinates is

$$D_{\mathrm{rot}} = T^T D T.$$

The orientation-dependent element stiffness is therefore

$$K_e(\rho_e, \alpha_e) = \rho_e^p \int_{\Omega_e} B^T D_{\mathrm{rot}}(\alpha_e) B \, d\Omega.$$

With quadrature,

$$K_e(\rho_e, \alpha_e) \approx \rho_e^p \sum_{q=1}^{Q} w_q B(r_q)^T T(\alpha_e)^T D T(\alpha_e) B(r_q) |\det J(r_q)|.$$

A.6 Orientation sensitivity

For one element, the compliance contribution is

$$C_e = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D T \varepsilon_q |\det J_q|, \qquad \varepsilon_q = B(r_q) u_e.$$

Only T depends on the orientation angles, so the derivative with respect to angle α_j is

$$\frac{\partial C_e}{\partial \alpha_j} = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T \left(\frac{\partial T^T}{\partial \alpha_j} D T + T^T D \frac{\partial T}{\partial \alpha_j} \right) \varepsilon_q |\det J_q|.$$

If D is symmetric, the scalar expression can also be evaluated as

$$\frac{\partial C_e}{\partial \alpha_j} = 2\rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D \frac{\partial T}{\partial \alpha_j} \varepsilon_q |\det J_q|,$$

provided the same transformation convention is used.

A.7 Rotation derivatives

The derivatives of the elementary rotation matrices are

$$\frac{\partial R_1}{\partial \alpha_1} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -\sin\alpha_1 & -\cos\alpha_1 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \end{bmatrix},$$

$$\frac{\partial R_2}{\partial \alpha_2} = \begin{bmatrix} -\sin\alpha_2 & 0 & \cos\alpha_2 \\ 0 & 0 & 0 \\ -\cos\alpha_2 & 0 & -\sin\alpha_2 \end{bmatrix},$$

$$\frac{\partial R_3}{\partial \alpha_3} = \begin{bmatrix} -\sin\alpha_3 & -\cos\alpha_3 & 0 \\ \cos\alpha_3 & -\sin\alpha_3 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

For $R = R_1 R_2 R_3$, the total rotation derivatives are

$$\frac{\partial R}{\partial \alpha_1} = \frac{\partial R_1}{\partial \alpha_1} R_2 R_3,$$

$$\frac{\partial R}{\partial \alpha_2} = R_1 \frac{\partial R_2}{\partial \alpha_2} R_3,$$
$$\frac{\partial R}{\partial \alpha_3} = R_1 R_2 \frac{\partial R_3}{\partial \alpha_3}.$$

The derivative $\partial T / \partial \alpha_j$ is obtained by differentiating every entry of $T(R)$ with the product rule. For example, an entry $R_{ab} R_{cd}$ contributes

$$\frac{\partial}{\partial \alpha_j} \left(R_{ab} R_{cd} \right) = \frac{\partial R_{ab}}{\partial \alpha_j} R_{cd} + R_{ab} \frac{\partial R_{cd}}{\partial \alpha_j}.$$

This product-rule structure is the key point of the orientation derivative: once R, $\partial R / \partial \alpha_j$, T, and $\partial T / \partial \alpha_j$ are known, the sensitivity follows directly from the quadrature expression above.